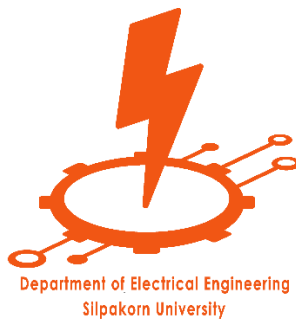


รายงานการปฏิบัติงานสหกิจศึกษา

การออกแบบและพัฒนา AI Assistant Chatbot
เพื่อช่วยตอบคำถามและแนะนำการใช้งาน
บนเว็บไซต์และแอปพลิเคชันของ Finansia Hero

นาย อชิรธีร์ อินยา 650910304

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา 618493 สหกิจศึกษา
สาขาวิชาวิศวกรรมอิเล็กทรอนิกส์และระบบคอมพิวเตอร์
ภาควิชาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์และเทคโนโลยีอุตสาหกรรม
มหาวิทยาลัยศิลปากร
ภาคการศึกษาต้น ปีการศึกษา 2568



รายงานการปฏิบัติงานสหกิจศึกษา

การออกแบบและพัฒนา AI Assistant Chatbot
เพื่อช่วยตอบคำถามและแนะนำการใช้งาน
บนเว็บไซต์และแอปพลิเคชันของ Finansia Hero

นาย อชิรธีร์ อินยา 650910304

ปฏิบัติงาน ณ

บริษัทหลักทรัพย์ ฟินันเซีย ไซรัส จำกัด (มหาชน)

เลขที่ 719 อาคารมินท์ ทาวเวอร์ ชั้น 6 วังใหม่ ปทุมวัน

กรุงเทพมหานคร 10330

บทคัดย่อ

บริษัทหลักทรัพย์ ฟินันเซีย ไซรัส จำกัด (มหาชน) เป็นบริษัทหลักทรัพย์ชั้นนำของประเทศไทย ที่ให้บริการด้านการลงทุนอย่างครบวงจร ทั้งในตลาดหลักทรัพย์และผลิตภัณฑ์ทางการเงินอื่น ๆ ภายใต้แพลตฟอร์ม Finansia Hero ซึ่งเป็นแอปพลิเคชันสำหรับการซื้อขายหลักทรัพย์

ในการปฏิบัติงานสหกิจศึกษา ข้าพเจ้าได้รับมอบหมายให้ฝึกในตำแหน่ง AI Engineer Intern โดยมีหน้าที่หลักในพัฒนา วิจัย และประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ (AI) เพื่อนำมาประยุกต์ใช้ในการยกระดับบริการ และขับเคลื่อนนวัตกรรมภายในองค์กร

โดยข้าพเจ้ามีหน้าที่หลักและรับผิดชอบในการออกแบบและพัฒนา AI Assistant Chatbot เพื่อช่วยตอบคำถามและแนะนำการใช้งานบนเว็บไซต์และแอปพลิเคชันของ Finansia Hero โครงการนี้มุ่งเน้นการนำเทคโนโลยีปัญญาประดิษฐ์และการประมวลผลภาษาธรรมชาติมาใช้เพื่อให้แชทบอทสามารถตอบคำถามทั่วไปเกี่ยวกับการใช้งานระบบ และให้คำแนะนำขั้นตอนต่าง ๆ แก่ผู้ใช้อย่างถูกต้องและรวดเร็ว ส่งผลให้บริษัทสามารถเพิ่มประสิทธิภาพในการให้บริการลูกค้า ลดภาระงานของเจ้าหน้าที่ และช่วยให้ผู้ใช้งานเข้าถึงข้อมูลได้สะดวกยิ่งขึ้น

Abstract

Finansia Syrus Securities Public Company Limited is a leading securities firm in Thailand that provides comprehensive investment services under the Finansia Hero platform, which offers both web-based and mobile trading applications. During my cooperative education program, I worked as an AI Engineer Intern, responsible for researching, developing, and applying artificial intelligence (AI) technologies to enhance services and drive innovation within the organization.

My primary responsibility was to design and develop an AI Assistant Chatbot to assist users by answering inquiries and providing usage guidance for the Finansia Hero platform on both the website and mobile application. The project aimed to integrate AI and Natural Language Processing (NLP) technologies, enabling the chatbot to understand user queries and respond accurately and efficiently. As a result, the company benefits from improved customer service efficiency, reduced workload for support staff, and a more convenient and accessible user experience.

กิตติกรรมประกาศ(ยังไม่เสร็จ)

การที่ข้าพเจ้าได้มาปฏิบัติงานสหกิจศึกษา ณ บริษัท เจวีซี แมนูแฟคเจอร์ริง (ประเทศไทย จำกัด) ตั้งแต่วันที่ 8 มกราคม พ.ศ.2561 ถึงวันที่ 11 พฤษภาคม พ.ศ.2561 นั้น ส่งผลให้ข้าพเจ้าได้รับความรู้ และประสบการณ์ต่างๆ ที่มีค่ามากมาย สำหรับรายงานสหกิจศึกษาฉบับนี้ สำเร็จลงได้ด้วยดี จากความร่วมมือ และสนับสนุนจากหลายฝ่าย ดังนี้

- | | | |
|----------------|---------------|-------------------------------------|
| 1. คุณ โฆษิต | ธรรมธาดา | Chief Technology Officer |
| 2. คุณ คุณิศรา | ศรีสุภาเดชา | Executive Secretary |
| 3. คุณ สุภาชาญ | ภาควิโรจน์กุล | IT Infrastructure & Project Manager |

แผนก IT AI

- | | | |
|---------------------|---------|-----------------|
| 4. คุณ ศักดิ์สิทธิ์ | กระแสน | Head of AI Team |
| 5. คุณ อภิวัฒน์ | ปานิเสน | Senior Engineer |
| 6. คุณ ธนภรณ์ | พิมาลัย | Senior Engineer |
| 7. คุณ ธนภรณ์ | พิมาลัย | Senior Engineer |
| 8. คุณ ธนภรณ์ | พิมาลัย | Senior Engineer |

และบุคคลท่านอื่นที่ไม่ได้กล่าวนามทุกท่านที่ได้ให้คำแนะนำช่วยเหลือในการจัดทำรายงานฉบับนี้

ข้าพเจ้าใคร่ขอขอบพระคุณผู้ที่มีส่วนเกี่ยวข้องทุกท่านที่มีส่วนในการให้ข้อมูล รวมทั้งเป็นที่ปรึกษาในการทำรายงานฉบับนี้จนเสร็จสมบูรณ์ ตลอดจนให้การดูแล และให้ความเข้าใจเกี่ยวกับชีวิตการทำงานจริง ข้าพเจ้าขอขอบคุณไว้ ณ ที่นี้

อชิรธีร์ อินยา

ผู้จัดทำรายงาน

20 ตุลาคม 2568

สารบัญ

	หน้า
บทคัดย่อภาษาไทย	ก
บทคัดย่อภาษาอังกฤษ	ข
กิตติกรรมประกาศ	ค
สารบัญ	ง
สารบัญตาราง	จ
สารบัญรูปภาพ	ฉ
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญ	1
1.2 วัตถุประสงค์	1
1.3 ขอบเขต	1
1.4 แผนการดำเนินงาน	2
1.5 ประโยชน์ที่คาดว่าจะได้รับ	3
บทที่ 2 ทฤษฎีที่เกี่ยวข้อง	4
2.1 ปัญญาประดิษฐ์	4
2.2 Visual Studio Code	4
2.3 ภาษา Python	5
2.4 Natural Language Processing	5
2.5 Retrieval-Augmented Generation (RAG)	6
2.6 Large Language Model (LLM)	7
2.7 AI Agent	7
2.8 อื่นๆ ที่เกี่ยวข้อง	7
บทที่ 3 วิธีการดำเนินงาน	8
3.1 การออกแบบและพัฒนาระบบ AI Assistant Chatbot	8
บทที่ 4 ผลการดำเนินงาน	12
บทที่ 5 สรุปผลและข้อเสนอแนะ	16
5.1 สรุปผล	x

5.2 ปัญหาและข้อเสนอแนะ	16
5.3 แนวทางพัฒนาต่อยอด	17
เอกสารอ้างอิง	18
ภาคผนวก ก	20
ภาคผนวก ข	22

สารบัญรูปภาพ

รูปที่	หน้า
2.1 กราฟพาวเวอร์โตนแสดง %Error แต่ละชนิดของเครื่องจักร	9
2.2 กราฟพาวเวอร์โตนแสดง %Error ของเครื่องจักร AV	10
2.3 แสดงภาพเครื่องจักร AV-8	11
3.1 เส้นแสดงความสัมพันธ์ระหว่างค่าพิสัยเฉลี่ย และค่าเฉลี่ยงานย่อย	62
3.2 XXXX	

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

บริษัทหลักทรัพย์ ฟินันเซีย ไซรัส จำกัด (มหาชน) เป็นบริษัทหลักทรัพย์ชั้นนำของประเทศไทย ที่ให้บริการด้านการลงทุนในตลาดหลักทรัพย์และผลิตภัณฑ์ทางการเงินต่าง ๆ ผ่านแพลตฟอร์ม Finansia Hero ซึ่งเป็นระบบซื้อขายหลักทรัพย์แบบครบวงจรทั้งบนเว็บไซต์และแอปพลิเคชันมือถือ

ในปัจจุบัน บริษัทมีฐานผู้ใช้งานจำนวนมากและมีความต้องการรับข้อมูลหรือความช่วยเหลือ ด้านการใช้งานระบบอยู่ตลอดเวลา การให้บริการผ่านเจ้าหน้าที่ฝ่ายลูกค้าสัมพันธ์เพียงอย่างเดียวอาจ ไม่สามารถตอบสนองความต้องการของผู้ใช้งานได้อย่างรวดเร็วเพียงพอ ดังนั้นจึงได้มีแนวคิดในการ พัฒนา AI Assistant Chatbot เพื่อช่วยตอบคำถามทั่วไปของผู้ใช้งาน แนะนำขั้นตอนการใช้งาน ระบบ และช่วยลดภาระของเจ้าหน้าที่ฝ่ายบริการลูกค้า

การพัฒนา AI Assistant Chatbot ในโปรเจกต์นี้มุ่งหวังให้ผู้ใช้งานสามารถเข้าถึงข้อมูลการใช้งาน และคำแนะนำต่าง ๆ ได้อย่างสะดวก รวดเร็ว และทันสมัยมากยิ่งขึ้น ซึ่งจะช่วยยกระดับ ประสิทธิภาพของผู้ใช้และเพิ่มประสิทธิภาพในการให้บริการขององค์กรโดยรวม

1.2 วัตถุประสงค์

1.2.1 เพื่อออกแบบและพัฒนา AI Assistant Chatbot สำหรับให้คำแนะนำผู้ใช้งาน

Finansia Hero

1.2.2 เพื่อศึกษาการประยุกต์ใช้เทคโนโลยี AI และ NLP ในการพัฒนา Chatbot

1.2.3 เพื่อลดภาระงานเจ้าหน้าที่บริการลูกค้า และเพิ่มประสิทธิภาพการบริการข้อมูล

1.3 ขอบเขต

1.3.1 พัฒนา AI Assistant Chatbot สำหรับแนะนำการใช้งานแอปพลิเคชัน Finansia Hero เพื่อช่วยตอบคำถามและแนะนำการใช้งานให้แก่ผู้ใช้งานทั่วไป

1.3.2 ใช้เทคโนโลยี Artificial Intelligence (AI) และ Natural Language Processing (NLP) ร่วมกับ Retrieval-Augmented Generation (RAG) และ AI Agent เพื่อเพิ่มประสิทธิภาพในการตอบคำถามให้มีความถูกต้องและเป็นธรรมชาติมากขึ้น

1.3.3 ใช้ข้อมูลจากเว็บไซต์และเอกสารคู่มือของ Finansia Hero เป็นแหล่งข้อมูลหลัก

1.3.4 ทดสอบระบบเบื้องต้นด้วยชุดคำถามตัวอย่างและคำถามจากพี่ๆ ในทีมพัฒนา เพื่อ
ประเมินความถูกต้องของคำตอบและการทำงานของระบบ

1.4 แผนการดำเนินงาน

ตารางที่ 1.1 แผนการดำเนินงาน

รายการ	สัปดาห์ที่															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1.ระดม แนวคิดและ กำหนด ขอบเขต โครงการ	↔															
2.เก็บ Requirement จากทีมที่ เกี่ยวข้อง			↔													
3.ออกแบบ สถาปัตยกรรม ระบบ						↔										
4. พัฒนาและ สร้างระบบ Chatbot							↔									
5.ทดสอบ ระบบและ ปรับปรุง ข้อผิดพลาด													↔			
6.จัดทำ รายงาน สรุปผล															↔	

1.5 ประโยชน์ที่คาดว่าจะได้รับ

1.5.1 ได้เรียนรู้และเข้าใจหลักการทำงานของเทคโนโลยีปัญญาประดิษฐ์ (AI) และการประมวลผลภาษาธรรมชาติ (NLP) ในการพัฒนา Chatbot

1.5.2 ได้ประสบการณ์ตรงในการเก็บ Requirement จากผู้เชี่ยวชาญหลายๆคน เพื่อนำมาพัฒนาและออกแบบระบบ Chatbot

1.5.3 ช่วยให้บริษัทสามารถให้บริการลูกค้าได้รวดเร็วและมีประสิทธิภาพมากยิ่งขึ้น

1.5.3 พัฒนาแนวทางสำหรับการนำ AI มาประยุกต์ใช้ในด้านบริการลูกค้าในอนาคต

บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

ในบทนี้จะกล่าวถึงทฤษฎีและแนวคิดพื้นฐานที่เกี่ยวข้องกับการออกแบบและพัฒนา AI Assistant Chatbot โดยเน้นที่เทคโนโลยีปัญญาประดิษฐ์ (Artificial Intelligence) การประมวลผลภาษาธรรมชาติ (Natural Language Processing) และเครื่องมือ/ภาษาที่ใช้ในการพัฒนาโครงการ

2.1 ปัญญาประดิษฐ์ (Artificial Intelligence: AI)

ปัญญาประดิษฐ์ หรือ AI คือ ศาสตร์และเทคโนโลยีที่เกี่ยวข้องกับการสร้างเครื่องจักรหรือโปรแกรมคอมพิวเตอร์ให้มีความสามารถในการคิด การเรียนรู้ การตัดสินใจ หรือการแก้ไขปัญหาได้เหมือนหรือใกล้เคียงกับความสามารถของมนุษย์ โดยมีวัตถุประสงค์เพื่อพัฒนาระบบที่สามารถทำงานได้อย่างชาญฉลาดในบริบทที่หลากหลาย

AI แบ่งออกเป็นหลายสาขาย่อย โดยสาขาที่เกี่ยวข้องกับโครงการนี้คือ:

2.1.1 การเรียนรู้ของเครื่อง (Machine Learning: ML)

Machine Learning (ML) เป็นสาขาย่อยของ AI ที่มุ่งเน้นการสร้างอัลกอริทึมที่ทำให้ระบบคอมพิวเตอร์สามารถ เรียนรู้จากข้อมูล (Data) โดยไม่จำเป็นต้องถูกเขียนโปรแกรมสั่งงานแบบเจาะจงในทุกขั้นตอน ระบบจะปรับปรุงประสิทธิภาพของตัวเองตามปริมาณข้อมูลและการฝึกฝน ซึ่งเป็นพื้นฐานสำคัญในการพัฒนาโมเดลสำหรับแชทบอท

2.1.2 การเรียนรู้เชิงลึก (Deep Learning: DL)

Deep Learning (DL) เป็นส่วนหนึ่งของ Machine Learning ที่ใช้โครงข่ายประสาทเทียม (Artificial Neural Networks) ที่มีชั้นซ่อน (Hidden Layers) จำนวนมาก (จึงเรียกว่า "Deep") DL มีความสามารถสูงในการเรียนรู้คุณลักษณะ (Features) ที่ซับซ้อนจากข้อมูลขนาดใหญ่ ซึ่งมักถูกนำมาใช้ในการพัฒนาโมเดลสำหรับการประมวลผลภาษาธรรมชาติ (NLP) ที่มีความแม่นยำสูง

2.2 Visual Studio Code (VS Code)

Visual Studio Code หรือ VS Code เป็นโปรแกรมแก้ไขโค้ด (Code Editor) จากบริษัทไมโครซอฟท์ ที่ได้รับความนิยมอย่างแพร่หลาย เนื่องจากเป็นซอฟต์แวร์แบบโอเพนซอร์ส (Open Source) ที่สามารถใช้งานได้ฟรี รองรับหลายระบบปฏิบัติการ เช่น Windows, macOS และ Linux

VS Code รองรับภาษาการเขียนโปรแกรมหลากหลาย เช่น Python, JavaScript, TypeScript, HTML, CSS และ PHP นอกจากนี้ยังมีส่วนขยาย (Extensions) สำหรับการพัฒนา

โปรแกรมด้าน AI, Machine Learning, และ Chatbot เช่น Jupyter, Python Extension, GitHub Copilot และอื่น ๆ ที่ช่วยให้การพัฒนาโค้ดสะดวกและมีประสิทธิภาพยิ่งขึ้น

2.3 ภาษา Python

ภาษา Python เป็นภาษาการเขียนโปรแกรมระดับสูงที่ได้รับความนิยมอย่างมากในวงการปัญญาประดิษฐ์ (Artificial Intelligence) และการประมวลผลภาษาธรรมชาติ (Natural Language Processing: NLP) เนื่องจากมีไลบรารีและเฟรมเวิร์กที่รองรับการพัฒนา AI ได้อย่างครบถ้วน เช่น

- **TensorFlow, PyTorch** – สำหรับการพัฒนาและฝึกโมเดล Machine Learning
- **LangChain, OpenAI API, Hugging Face Transformers** – สำหรับพัฒนา Chatbot และ AI Agent
- **Flask, FastAPI** – สำหรับสร้าง Web API ที่เชื่อมต่อกับระบบภายนอก
- **Libraries พื้นฐาน** เช่น
 - requests สำหรับเชื่อมต่อ API
 - os และ dotenv สำหรับจัดการค่าคอนฟิก
 - json, re, datetime สำหรับประมวลผลข้อมูลพื้นฐาน

Python มีจุดเด่นด้านโครงสร้างภาษาที่อ่านง่าย ทำให้เหมาะสำหรับงานวิจัย พัฒนาโปรโตไทป์ และการนำ AI ไปใช้งานจริงในองค์กร และเป็นหัวใจสำคัญของการพัฒนาโครงการนี้ทั้งในด้านการสร้างระบบเบื้องหลัง การเชื่อมต่อฐานข้อมูล และการจัดการกระบวนการ RAG ทั้งหมด

2.4 Natural Language Processing (NLP)

การประมวลผลภาษาธรรมชาติ (Natural Language Processing: NLP) เป็นเทคโนโลยีที่ช่วยให้คอมพิวเตอร์สามารถเข้าใจและตีความภาษามนุษย์ได้ ทั้งในรูปแบบข้อความหรือเสียง การนำ NLP มาใช้ใน Chatbot ช่วยให้ระบบสามารถทำความเข้าใจคำถามของผู้ใช้ (Intent Recognition) และดึงข้อมูลที่เกี่ยวข้องเพื่อตอบกลับอย่างเหมาะสม

เทคนิคที่ใช้ใน NLP ได้แก่

- **Tokenization** – การแยกข้อความออกเป็นหน่วยย่อย
- **Named Entity Recognition (NER)** – การตรวจจับชื่อบุคคล องค์กร หรือหน่วยข้อมูลสำคัญ
- **Semantic Search** – การค้นหาข้อมูลตามความหมายแทนที่จะค้นตามคำตรงตัว
- **Language Model** – โมเดลที่เรียนรู้ความสัมพันธ์ของคำ เช่น GPT, BERT, หรือ LLaMA

2.4 Retrieval-Augmented Generation (RAG)

RAG ย่อมาจาก Retrieval-Augmented Generation เป็นเทคนิคที่ผสมผสานการดึงข้อมูล (Retrieval) และการสร้างข้อความ (Generation) เข้าด้วยกัน เพื่อเพิ่มความถูกต้องและทันสมัยให้กับโมเดลภาษาขนาดใหญ่ (LLM) โดยการผสมผสานการค้นคืนข้อมูลจากแหล่งภายนอกเข้ากับกระบวนการสร้างคำตอบ แม้ว่า LLMs แบบดั้งเดิมจะมีประสิทธิภาพสูง แต่ก็มีข้อจำกัดที่ข้อมูลที่ใช้ในการฝึกฝน ซึ่งอาจล้าสมัยหรือขาดความรู้เฉพาะทางอย่างรวดเร็ว RAG แก้ไขข้อจำกัดเหล่านี้โดยการนำข้อมูลที่ทันสมัยและเฉพาะทางจากแหล่งภายนอกมาใช้ ทำให้โมเดลสามารถสร้างคำตอบที่ทั้งทันสมัยและตรงกับบริบทได้

RAG ในโครงการนี้ประกอบด้วยองค์ประกอบหลักดังนี้:

2.4.1 Retrieval (การค้นคืนข้อมูล)

เป็นกระบวนการค้นหาข้อมูลที่เกี่ยวข้องกับคำถามของผู้ใช้จากฐานข้อมูล โดยใช้ **Semantic Search** ซึ่งวัด “ความใกล้เคียงทางความหมาย” แทนการจับคำตรงตัว เพื่อให้ระบบสามารถดึงข้อมูลที่มีเนื้อหาเกี่ยวข้องได้แม่นยำขึ้น

2.4.2 Embedding

ก่อนจะค้นหาได้ ต้องแปลงข้อความทั้ง “คำถาม” และ “เอกสาร” ให้เป็นเวกเตอร์ตัวเลข (Vector) ด้วยโมเดลที่เรียกว่า **Embedding Model** ในโครงการนี้ใช้โมเดล **BGE-M3** ซึ่งสามารถรองรับได้หลายภาษา รวมถึงภาษาไทย เหมาะสำหรับงาน Semantic Search และ Retrieval

2.4.3 Vector Database

ข้อมูลที่ผ่านการแปลงเป็นเวกเตอร์จะถูกจัดเก็บในฐานข้อมูลเฉพาะทางที่เรียกว่า **Vector Database** ในโครงการนี้ใช้ **Qdrant** ซึ่งมีประสิทธิภาพสูงในการค้นหาเวกเตอร์ที่มีความใกล้เคียงกัน (Nearest Neighbor Search) ด้วยอัลกอริทึม HNSW ระบบสามารถค้นหาเอกสารที่ใกล้เคียงกับคำถามได้อย่างรวดเร็ว โดยเฉลี่ยใช้เวลาเพียงไม่กี่มิลลิวินาที

2.4.4 Context Assembly & Generation

เมื่อระบบค้นคืนเอกสารที่เกี่ยวข้องได้แล้ว จะนำเนื้อหานั้นไปเป็น “บริบท” (Context) ให้กับโมเดลภาษา (LLM) เพื่อสร้างคำตอบที่อ้างอิงจากข้อมูลจริง ซึ่งกระบวนการนี้ช่วยให้ Chatbot ตอบได้แม่นยำและสอดคล้องกับเนื้อหาจริงมากขึ้น

2.5 Large Language Model (LLM)

LLM (Large Language Model) คือโมเดลภาษาขนาดใหญ่ที่ผ่านการฝึกจากข้อมูลจำนวนมาก เพื่อให้สามารถเข้าใจและสร้างข้อความภาษามนุษย์ได้อย่างเป็นธรรมชาติ บทบาทของ LLM ในโครงการนี้คือ

- ทำความเข้าใจคำถามของผู้ใช้ (Intent Detection)
- ผสมผสานข้อมูลจาก RAG เพื่อสร้างคำตอบ
- ปฏิบัติตาม **System Prompt** ที่ควบคุมทิศทางการตอบ เช่น ให้ตอบเฉพาะข้อมูลจาก Finasia Hero และปฏิเสธเมื่อไม่มีข้อมูล

จุดเด่นของ LLM คือมีความสามารถในการ “เข้าใจความหมาย” ของคำถามที่ซับซ้อน และสามารถสร้างคำตอบที่มีโครงสร้างและถ้อยคำใกล้เคียงกับมนุษย์มากกว่าระบบ Chatbot แบบ rule-based

2.6 AI Agent

AI Agent เป็นโมดูลอัจฉริยะที่ช่วยบริหารการทำงานของระบบ Chatbot โดยทำหน้าที่ควบคุมลำดับขั้นตอน เช่น การตัดสินใจว่าจะเรียกใช้ RAG หรือไม่ การวิเคราะห์ประเภทของคำถาม และการตรวจสอบผลลัพธ์ก่อนส่งให้ผู้ใช้

กระบวนการทำงานของ Agent มีขั้นตอนหลักดังนี้

1. รับคำถามจากผู้ใช้และวิเคราะห์เจตนา (Intent)
2. เลือกกลไกที่เหมาะสม เช่น ดึงข้อมูลผ่าน RAG หรือให้ LLM ตอบตรง
3. รวมผลลัพธ์ ตรวจสอบความถูกต้อง และปรับรูปแบบคำตอบ
4. ส่งผลลัพธ์ให้ผู้ใช้ในรูปแบบที่อ่านเข้าใจง่าย

Agent ช่วยให้ระบบ Chatbot มีความยืดหยุ่นและสามารถทำงานหลายขั้นตอนในรอบการสนทนาเดียวได้ โดยเฉพาะเมื่อคำถามมีความซับซ้อนหรือเกี่ยวข้องกับข้อมูลหลายประเภท

2.7 อื่นๆ ที่เกี่ยวข้อง

- Framework ที่เกี่ยวข้องกับการพัฒนา Chatbot เช่น LangChain, LangGraph, etc.
- **Flask / FastAPI** – ใช้สร้าง REST API สำหรับเชื่อมต่อโมเดลกับส่วนหน้า (Frontend)
- **Gemini API** – ใช้เรียกใช้งานโมเดลภาษา LLM ที่รองรับการตอบสนองตามบริบท

บทที่ 3

วิธีการดำเนินงาน

3.1 การออกแบบและพัฒนาระบบ AI Assistant Chatbot

การดำเนินงานของโครงการนี้เริ่มจากการศึกษาและรวบรวมข้อมูลของระบบ **Finansia Hero** เพื่อออกแบบและพัฒนา **AI Assistant Chatbot** ที่สามารถช่วยตอบคำถามและแนะนำการใช้งานระบบได้อย่างถูกต้อง โดยมีขั้นตอนการทำงานหลักดังต่อไปนี้

3.1.1 การวิเคราะห์และเก็บรวบรวมความต้องการ (Requirement Analysis)

ในขั้นตอนแรก ได้ทำการศึกษาระบบ **Finansia Hero** ทั้งในรูปแบบเว็บไซต์และแอปพลิเคชัน เพื่อทำความเข้าใจฟังก์ชันการใช้งาน เช่น การเปิดบัญชี การเข้าสู่ระบบ การซื้อขายหุ้น และการใช้งานเมนูต่างๆ จากนั้นจึงรวบรวมความต้องการของผู้ใช้งานร่วมกับทีมที่ปรึกษา เพื่อกำหนดขอบเขตของ Chatbot ว่าจะสามารถตอบคำถามในหัวข้อใดได้บ้าง เช่น

- การใช้งานเบื้องต้นของระบบ
- การแก้ไขปัญหาที่พบบ่อย
- การแนะนำขั้นตอนต่างๆ ภายในแอป

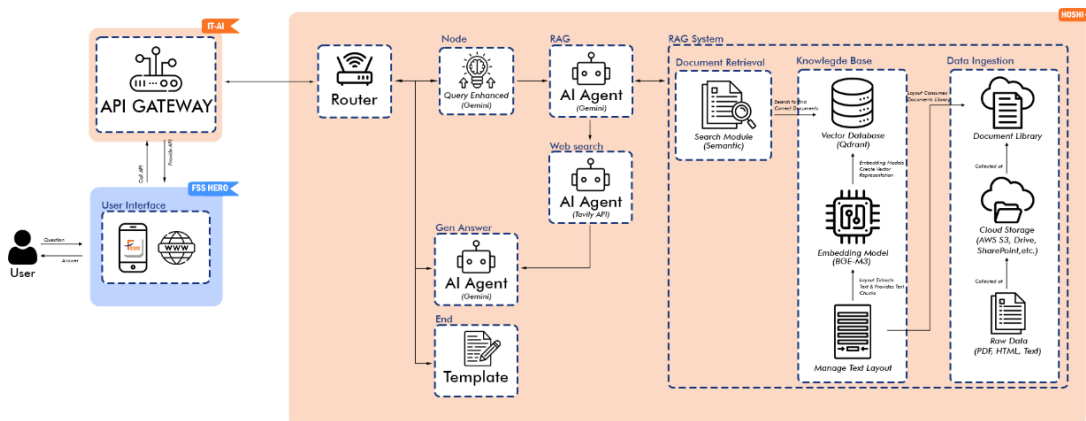
ผลลัพธ์ที่ได้จากขั้นตอนนี้คือ **รายการคำถาม-คำตอบ (FAQ list)** และ **ขอบเขตการให้บริการของ Chatbot**

3.1.2 การออกแบบสถาปัตยกรรมระบบ (System Architecture Design)

สถาปัตยกรรมระบบของโครงการนี้อ้างอิงจากแผนภาพการทำงานจริง (ดังรูป) โดยแบ่งออกเป็นหลายโมดูลที่ทำงานร่วมกันดังนี้

- **User Interface (Frontend)**
 - ส่วนติดต่อผู้ใช้ ที่อยู่บนเว็บไซต์และแอปพลิเคชัน Finansia Hero
 - ผู้ใช้สามารถพิมพ์คำถามหรือปัญหา ระบบจะส่งข้อมูลเข้าสู่ API Gateway
- **API Gateway & Router (Backend Core)**
 - ทำหน้าที่เป็นจุดเชื่อมต่อระหว่างแอปพลิเคชันของ Finansia Hero และระบบ AI Chatbot
 - รับข้อความจากผู้ใช้และส่งต่อไปให้ Node ประมวลผลผ่าน AI Agent
- **Node Layer / Query Processing**

- ใช้ **AI Agent (Gemini)** ทำหน้าที่ประมวลผลคำถามและปรับปรุงรูปแบบคำถาม (Query Enhancement) ก่อนเข้าสู่ระบบ RAG
- หากไม่พบข้อมูลในฐานความรู้ ระบบสามารถเรียกใช้งาน “Web Search Agent” เพื่อค้นข้อมูลภายนอกที่เกี่ยวข้องได้
- **RAG System (Retrieval-Augmented Generation)**
 - ส่วนสำคัญที่ใช้ค้นคืนข้อมูลและสร้างคำตอบ
 - ประกอบด้วย 3 ส่วนหลัก
 - **Document Retrieval:** ใช้การค้นหาแบบ Semantic Search เพื่อค้นหาเอกสารที่มีความหมายใกล้เคียงกับคำถาม
 - **Knowledge Base:** ใช้ Qdrant Vector Database สำหรับจัดเก็บเวกเตอร์ของเอกสาร
 - **Embedding Model (BGE-M3):** แปลงข้อความเป็นเวกเตอร์เพื่อใช้ในการค้นหาและเปรียบเทียบความใกล้เคียงทางความหมาย
- **AI Agent & LLM (Gemini Model)**
 - ใช้ **Gemini (LLM)** เป็นโมเดลภาษาหลักในการประมวลผลคำถามและสร้างคำตอบ โดยทำงานร่วมกับ RAG
 - AI Agent จะเลือกกระบวนการที่เหมาะสม เช่น ดึงข้อมูลจากฐานความรู้, สร้างคำตอบด้วยโมเดล, หรือผสมทั้งสองแบบ
- **Output & Template Generation**
 - หลังจาก AI Agent สร้างคำตอบแล้ว ระบบจะจัดรูปแบบให้อ่านง่าย ก่อนส่งกลับไปแสดงผลที่หน้าจอผู้ใช้ผ่านแอปพลิเคชัน



รูปที่ 3.1 แผนภาพแสดงความสัมพันธ์ (System Architecture Diagram)

3.1.2 การพัฒนาระบบ (Development and Implementation)

การพัฒนา Chatbot ดำเนินการโดยใช้ ภาษา Python เป็นหลัก พร้อม Framework และเครื่องมือที่เกี่ยวข้องกับการสร้างระบบ RAG ดังนี้

- **LangChain:** ใช้จัดการลำดับการทำงาน (Chain) ระหว่าง RAG และ LLM
- **LangGraph:** ใช้สร้าง Flow เชิงโครงสร้างของ Agent และการควบคุมลำดับการประมวลผล
- **Qdrant:** เป็น Vector Database สำหรับจัดเก็บข้อมูลเวกเตอร์ที่สร้างจาก Embedding Model
- **BGE-M3:** ใช้เป็น Embedding Model สำหรับแปลงข้อความให้เป็นเวกเตอร์ที่ใช้ในกระบวนการค้นคืน
- **Gemini (LLM):** เป็นโมเดลภาษาหลักสำหรับสร้างคำตอบ โดยเชื่อมต่อผ่าน API
- **Streamlit:** ใช้สร้างหน้า Demo เพื่อสาธิตการทำงานของ Chatbot
- **Flask/FastAPI:** ใช้สำหรับสร้าง API ที่เชื่อมระหว่างระบบ Chatbot และ Frontend
- **Other Libraries:** เช่น requests, dotenv, json, os, pandas, และ faiss สำหรับการจัดการข้อมูลและเชื่อมต่อระบบ

ขั้นตอนการพัฒนาแบ่งออกเป็นลำดับดังนี้

- 1) **Data Collection** – ดึงข้อมูลจากเว็บไซต์ Finansia Hero
- 2) **Data Preparation & Embedding** – แปลงข้อมูลเป็นเวกเตอร์ด้วย BGE-M3
- 3) **Vector Storage** – จัดเก็บในฐานข้อมูล Qdrant
- 4) **RAG Pipeline Setup** – สร้างกระบวนการค้นคืนและสร้างคำตอบ
- 5) **AI Agent Integration** – ผสานการทำงานของ Gemini ให้เลือกใช้ retrieval หรือ generation ตามบริบท
- 6) **Frontend & Demo** – สร้างหน้า Streamlit เพื่อสาธิตการทำงานของระบบ

3.1.3 การทดสอบระบบ (Testing)

มีการทดสอบระบบด้วยชุดข้อมูลจำลองและคำถามจริงจากพี่ ๆ ในทีม เพื่อวัดความถูกต้องของคำตอบและประสิทธิภาพการค้นคืนข้อมูล โดยประเมินในด้าน

- ความถูกต้องของข้อมูล (Accuracy)
- ความเร็วในการตอบคำถาม (Response Time)
- ความเป็นธรรมชาติของภาษา (Fluency)
- ความสอดคล้องกับเนื้อหาจริงตามข้อมูลใน Knowledge Base

3.2 การสาธิตและการนำเสนอระบบ (System Demonstration)

3.2.1 การสร้างหน้า Demo ด้วย Streamlit

ใช้ **Streamlit** สร้างอินเทอร์เฟซสำหรับสาธิตการทำงานของ Chatbot โดยผู้ใช้สามารถพิมพ์คำถามและรับคำตอบแบบเรียลไทม์จากโมเดล Gemini ที่ผสานกับระบบ RAG

3.2.2 การนำเสนอระบบ

ได้จัดสาธิตให้ทีม IT-AI และฝ่ายที่เกี่ยวข้อง เพื่อแสดงความสามารถของระบบในการตอบคำถามและให้คำแนะนำการใช้งานแอป Finansia Hero แบบอัตโนมัติ

บทที่ 4

ผลการดำเนินงาน

รายละเอียดผลการดำเนินงานในโครงการนี้ ประกอบด้วยกระบวนการทดสอบและประเมินผลระบบ AI Assistant Chatbot ที่พัฒนาโดยใช้เทคโนโลยี RAG (Retrieval-Augmented Generation) และโมเดลภาษา Gemini เพื่อตอบคำถามจากผู้ใช้ โดยจะนำเสนอผลการดำเนินงานในด้านต่าง ๆ ดังต่อไปนี้

4.1 เงื่อนไขการทดลอง

การทดสอบและประเมินผลถูกออกแบบขึ้นเพื่อให้ผลลัพธ์มีความน่าเชื่อถือและสะท้อนประสิทธิภาพของระบบจริง โดยมีเงื่อนไขการทดลองดังนี้

4.1.1 ข้อมูลที่ใช้ในการทดสอบ

- ใช้ข้อมูลจริงจาก เว็บไซต์และคู่มือการใช้งานของ Finansia Hero
- ใช้คำถามจากพี่ ๆ ในทีม IT-AI และทีมพัฒนา เพื่อประเมินความถูกต้องและความสมจริงของคำตอบ (ทดสอบผ่าน **Streamlit Demo Interface**)

4.1.2 รูปแบบการทดสอบ

- ทดสอบด้วยคำถามตัวอย่าง (Sample Questions) ที่จำลองพฤติกรรมของผู้ใช้จริง
- ไม่มีการใช้ข้อมูลภายในหรือข้อมูลลูกค้าจริง เพื่อคงความปลอดภัยของข้อมูล

4.1.3 เทคโนโลยีที่ใช้ในการทดลอง

- LLM: Gemini
- Embedding Model: BGE-M3
- Vector Database: Qdrant
- Frameworks: LangChain, LangGraph, Flask, Streamlit
- ชุดข้อมูล: เอกสารคู่มือประเภท PDF, HTML, CSV และข้อความ (Text) เป็นข้อมูลจากเว็บไซต์ Finansia Hero

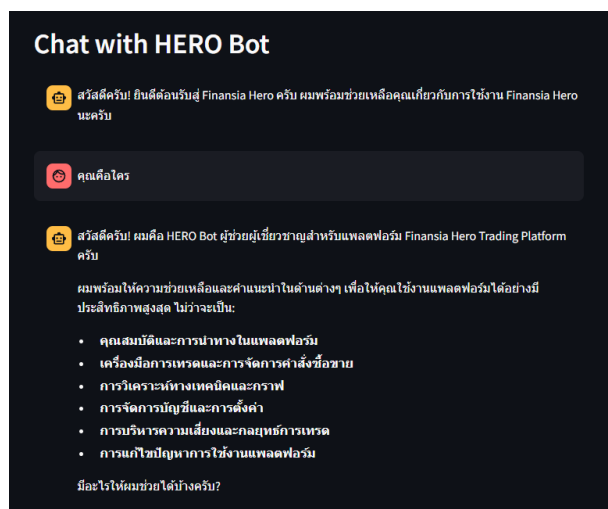
4.1.4 เกณฑ์การประเมินผล

- ความถูกต้องของคำตอบ (Answer Accuracy)
- ความสอดคล้องกับข้อมูลจริง (Context Relevance)
- ความเร็วในการตอบ (Response Time)
- ความเป็นธรรมชาติของภาษา (Fluency)

4.2 ผลการทดลอง

4.1.1 กรณีที่ 1: การทดสอบคำถามจากชุดข้อมูลตัวอย่าง (Sample Dataset)

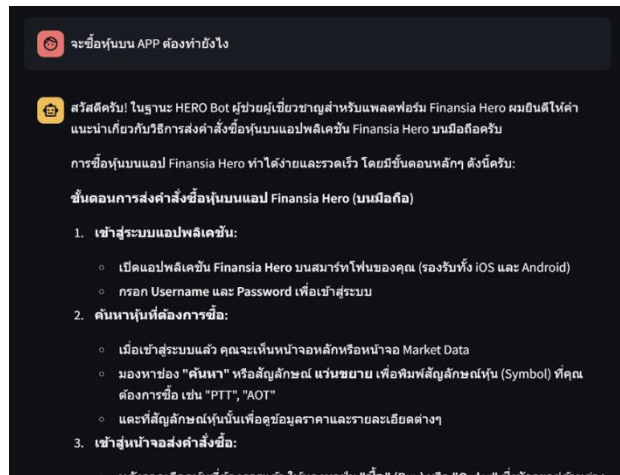
จากการทดสอบได้ทำการทดสอบกับคำถามที่สร้างขึ้นจากหัวข้อในคู่มือการใช้งาน ซึ่งแสดงรายละเอียดในช่องแชท ตัวอย่างในรูป



รูปที่ 4.1 ตัวอย่างการสนทนาเริ่มต้นกับ HERO Bot

4.2.2 กรณีที่ 2: การทดสอบคำถามจริงจากผู้ใช้งานภายในทีม

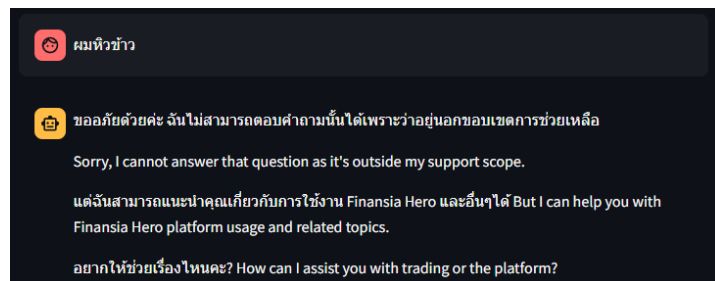
ได้มีการรวบรวมคำถามจากพี่ ๆ ในทีมพัฒนา IT-AI เพื่อใช้ทดสอบความสามารถของระบบในสถานการณ์จริงตัวอย่างในรูป



รูปที่ 4.2 ตัวอย่างการสนทนาที่มาจากพีๆในทีม IT AI

4.2.3 กรณีที่ 3: การทดสอบคำถามที่อยู่นอกเหนือจากขอบเขตข้อมูลที่กำหนด

ระบบได้ถูกออกแบบให้มี เงื่อนไขตรวจจับคำถามที่อยู่นอกขอบเขต (Out-of-bound) เช่น คำถามที่ไม่เกี่ยวข้องกับ Finasia Hero หรือคำถามที่เป็นเชิงสนทนาไม่เกี่ยวกับระบบ ในกรณีดังกล่าว ระบบจะตอบกลับโดยใช้ข้อความปฏิเสธที่กำหนดไว้ล่วงหน้า เช่น



รูปที่ 4.3 ตัวอย่างการสนทนาที่อยู่นอกเหนือจากขอบเขตข้อมูลที่กำหนด

4.3 เปรียบเทียบผลการทดลอง

รายการทดสอบ	จำนวน คำถาม	ความ ถูกต้อง	เฉลี่ย เวลา ตอบ (วินาที)	หมายเหตุ
ชุดคำถามตัวอย่าง (Sample Dataset)	50	92%	1.7	ระบบตอบได้ครบถ้วนและ สอดคล้องกับข้อมูลจริง
คำถามจากทีม ภายใน	35	87%	1.9	มีบางคำถามที่อยู่นอกขอบเขต ข้อมูลฐานความรู้
คำถามทั่วไป (การ สนทนาแบบเปิด)	20	95%	2.1	ระบบตอบได้ถูกต้องตามที่ กำหนดไว้ล่วงหน้า
คำถามนอก ขอบเขต (Out-of- Scope)	5	98%	1	ระบบตอบปฏิเสธตามข้อความ ที่กำหนดไว้ที่บ้าน

4.4 ผลลัพธ์ภาพรวมของระบบ

จากการประเมินผลทั้งหมด ระบบ AI Assistant Chatbot สามารถ

- ทำงานร่วมกับโครงสร้าง RAG ได้อย่างมีประสิทธิภาพ
- ลดภาระงานของฝ่ายบริการลูกค้าในการตอบคำถามซ้ำ ๆ
- สามารถปรับใช้ในระบบจริงของ Finansia Hero ได้โดยไม่กระทบต่อข้อมูลลูกค้า

นอกจากนี้ ระบบยังมีความยืดหยุ่น สามารถต่อยอดเพื่อเชื่อมกับฐานข้อมูลภายในหรือระบบอื่นในอนาคตได้ เช่น การตอบคำถามเกี่ยวกับการเทรดแบบเรียลไทม์ หรือการอัปเดตข้อมูลตลาดหุ้น

บทที่ 5

สรุปผลและข้อเสนอแนะ

5.1 สรุปผล

จากการปฏิบัติงานสหกิจศึกษาในโครงการ “การออกแบบและพัฒนา AI Assistant Chatbot เพื่อช่วยตอบคำถามและแนะนำการใช้งานบนเว็บไซต์และแอปพลิเคชันของ Finansia Hero” ข้าพเจ้าได้ดำเนินการครบตามวัตถุประสงค์ของโครงการที่ตั้งไว้ โดยสามารถออกแบบและพัฒนา ระบบ Chatbot ที่ทำงานบนสถาปัตยกรรม RAG (Retrieval-Augmented Generation) ร่วมกับโมเดลภาษา Gemini (LLM) ได้สำเร็จ

ระบบที่พัฒนาขึ้นสามารถ:

- ตอบคำถามได้ตรงกับข้อมูลจริง จากเว็บไซต์และคู่มือการใช้งานของ Finansia Hero
- ประมวลผลรวดเร็วและมีความแม่นยำสูง โดยเฉลี่ยตอบกลับภายใน 2 วินาที
- ตรวจจับคำถามนอกขอบเขตได้อัตโนมัติ ตอบกลับด้วยข้อความที่เหมาะสมตามที่ตั้งค่าไว้
- สามารถนำไปประยุกต์ใช้จริงเพื่อช่วยลดภาระงานของฝ่ายบริการลูกค้า (Customer Support)

5.2 ปัญหาและข้อเสนอแนะ

5.2.1 ปัญหาที่พบระหว่างดำเนินงาน

- ข้อจำกัดของข้อมูล: เนื่องจากข้อมูลที่ใช้มาจากเว็บไซต์และคู่มือการใช้งานเท่านั้น ทำให้บางหัวข้อยังไม่ครอบคลุมทุกกรณีที่ผู้ใช้อาจถาม
- ภาษารธรรมชาติของผู้ใช้ (User Input): ผู้ใช้บางคนพิมพ์คำถามในรูปแบบภาษาพูดหรือคำย่อ ทำให้ระบบเข้าใจผิดได้บ้างในบางกรณี
- เวลาประมวลผลในบางกรณี: คำถามที่ซับซ้อนและยาวอาจใช้เวลาในการค้นคืนข้อมูลและประมวลผลเพิ่มขึ้นเล็กน้อย

5.2.1 ข้อเสนอแนะเพื่อปรับปรุง

- ควรเพิ่มแหล่งข้อมูลภายใน (Internal Knowledge Base) เช่น คู่มือภายใน หรือเอกสารอบรม เพื่อให้ระบบตอบคำถามได้ครอบคลุมมากขึ้น
- พัฒนาโมดูล Intent Classification เพิ่มเติม เพื่อแยกประเภทคำถามของผู้ใช้ได้แม่นยำกว่าเดิม
- เพิ่มระบบ Feedback จากผู้ใช้ เพื่อเก็บข้อมูลคำถามที่ตอบผิดหรือไม่พอใจ เพื่อนำมาปรับปรุงการตอบในรุ่นถัดไป

5.3 แนวทางพัฒนาต่อยอด

คาดว่าจะสามารถดำเนินการพัฒนาต่อยอดได้ในทิศทางต่อไปนี้:

- **เพิ่มความสามารถในการประมวลผลเอกสาร (Document Processing)** เพื่อให้ระบบสามารถสกัดข้อมูลเชิงลึกและตอบคำถามได้แม่นยำยิ่งขึ้น โดยปรับปรุงกระบวนการประมวลผลไฟล์และจัดโครงสร้างข้อมูลก่อนเข้าสู่ขั้นตอนการค้นคืน (Retrieval)
- **พัฒนาการจดจำบริบทการสนทนา (Context Memory)** เพื่อให้ Chatbot สามารถจดจำคำถามก่อนหน้าและตอบต่อเนื่องได้อย่างเป็นธรรมชาติ ทำให้ประสบการณ์การใช้งานของผู้ใช้ราบรื่นและเป็นมิตรมากขึ้น
- **จัดเก็บและวิเคราะห์ประวัติการสนทนา (Chat History Analysis)** เพื่อใช้ข้อมูลจริงจากการสนทนาในการปรับปรุงประสิทธิภาพของระบบ เช่น วิเคราะห์คำถามที่ตอบผิด หรือหัวข้อที่ผู้ใช้สนใจมากที่สุด
- **เพิ่มระบบตรวจสอบและรับข้อเสนอแนะแบบเรียลไทม์ (Monitoring & Feedback System)** เพื่อให้ทีมพัฒนาสามารถติดตามคุณภาพของคำตอบ ประเมินความแม่นยำ และปรับปรุงระบบได้อย่างต่อเนื่อง

เอกสารอ้างอิง

- [1] LangChain AI, *LangGraph Documentation: Multi-Agent Workflows*, 2024. [Online]. Available: <https://langchain-ai.github.io/langgraph/>. [Accessed: Aug. 2025].
- [2] LangChain, *LangChain Framework Documentation*, 2024. [Online]. Available: <https://python.langchain.com/docs/>. [Accessed: Aug. 2025].
- [3] Google AI, *Gemini API Documentation*, 2025. [Online]. Available: <https://ai.google.dev/>. [Accessed: Aug. 2025].
- [4] Qdrant, *Qdrant Vector Database Documentation*, 2024. [Online]. Available: <https://qdrant.tech/documentation/>. [Accessed: Oct. 2025].
- [5] Beijing Academy of Artificial Intelligence, *FlagEmbedding: BGE-M3 Multilingual Embeddings*, 2024. [Online]. Available: <https://github.com/FlagOpen/FlagEmbedding>. [Accessed: Aug. 2025].
- [6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” 2019. [Online]. Available: <https://www.sbert.net/>. [Accessed: Aug. 2025].
- [7] P. Lewis, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *arXiv preprint*, arXiv:2005.11401, 2020.
- [8] A. Paszke, et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” 2019. [Online]. Available: <https://pytorch.org/>. [Accessed: Aug. 2025].
- [9] Hugging Face, *Transformers: State-of-the-art Machine Learning for PyTorch and TensorFlow*, 2024. [Online]. Available: <https://huggingface.co/docs/transformers/>. [Accessed: Aug. 2025].
- [10] Tavily AI, *Tavily Search API Documentation*, 2024. [Online]. Available: <https://tavily.com/>. [Accessed: Aug. 2025].
- [11] FastAPI, *FastAPI Framework Documentation*, 2024. [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Aug. 2025].
- [12] Streamlit, *Streamlit Documentation: Build Data Apps*, 2024. [Online]. Available: <https://docs.streamlit.io/>. [Accessed: Aug. 2025].

- [13] PyPDF Development Team, *PyPDF Documentation*, 2024. [Online]. Available: <https://pypdf.readthedocs.io/>. [Accessed: Aug. 2025].
- [14] Unstructured.io, *Unstructured Document Processing Library*, 2024. [Online]. Available: <https://unstructured.io/>. [Accessed: Aug. 2025].
- [15] Finansia Syrus Securities Public Company Limited, *Finansia HERO Trading Platform Documentation*, 2025.

หมายเหตุ:

- รูปแบบนี้เป็นไปตามมาตรฐานการอ้างอิงแบบ IEEE
- แหล่งข้อมูลทั้งหมดเป็น แหล่งจริง (official documentation / academic paper) ที่เกี่ยวข้องกับโครงงาน Chatbot ของคุณโดยตรง
- คุณสามารถคัดลอกวางลงในรายงาน Word ได้เลยโดยไม่ต้องปรับรูปแบบเพิ่มเติม

ภาคผนวก ก
ตัวอย่างชุดคำถาม

ภาคผนวก ก

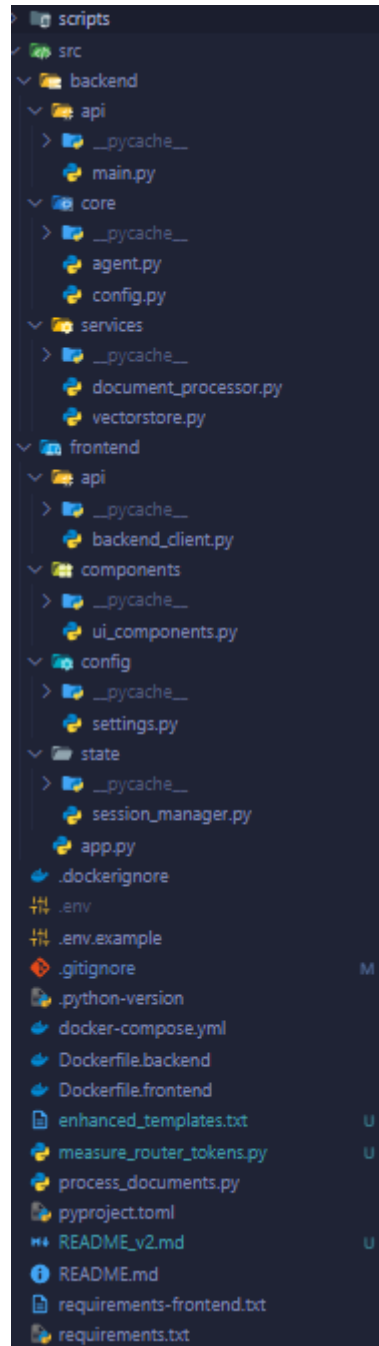
	A	B	C	D
1	คำถามตัวอย่างสำหรับ Chatbot (Finansia HERO)			
2	สมัครใช้งาน Finansia HERO ยังไงคะ?			
3	เข้าแอป HERO ไม่ได้ มันขึ้นว่า invalid user ต้องทำยังไงครับ			
4	ลืมรหัสผ่าน HERO รีเซ็ตยังไงคะ			
5	ล็อกอินแล้วแอปค้าง ต้องแก้ยังไงครับ			
6	เข้าแอปแล้วเดจออก ทำไมถึงเป็นแบบนี้			
7	ต้องเปิดพอร์ตกับ Finansia ก่อนถึงจะใช้ HERO ได้ไหม			
8	เปิดบัญชีใหม่กับ Finansia เพื่อใช้ HERO ใช้เวลากี่วัน			
9	Finansia HERO ใช้บัญชีเดิมจาก Streaming ได้ไหม			
10	ต้องยืนยันตัวตนยังไงถึงจะเข้าใช้ HERO ได้			
11	สมัคร HERO ผ่านหน้าเว็บได้ไหมคะ			
12	Finansia HERO คืออะไร มีไว้ทำอะไร			
13	แอป HERO ใช้ได้ทั้งในมือถือและคอมไหม			
14	HERO รองรับระบบ Android เวอร์ชันไหนบ้าง			
15	ใช้ HERO บน iPhone ต้องอัปเดต iOS ไหม			
16	มีค่าใช้จ่ายรายเดือนในการใช้ HERO หรือเปล่า			
17	HERO ใช้งานได้กี่เครื่องพร้อมกัน			
18	แอป HERO มีภาษาอังกฤษไหม			
19	HERO ต้องอัปเดตบ่อยไหมครับ			
20	เข้า HERO ผ่านเว็บได้ไหม หรือมีแค่แอป			
21	ใช้ HERO ในแท็บเล็ตได้ไหม			
22	ซื้อหุ้นใน HERO ยังไงคะ			
23	ทำไมกดซื้อหุ้นแล้วไม่ขึ้นในพอร์ต			
24	สามารถเทรด DW ผ่าน HERO ได้ไหม			
25	HERO รองรับการเทรดอนุพันธ์ไหม			
26	ตั้งคำสั่งซื้อแบบ Market Order ได้ไหม			
27	ส่งคำสั่งซื้อแล้วสถานะขึ้น Pending ต้องรอนานไหม			
28	HERO รองรับ Odd Lot หรือไม่			
29	สามารถตั้ง Stop Loss ได้ไหมใน HERO			
30	ตั้ง Take Profit ยังไงใน HERO			
31	คำสั่งซื้อขายใน HERO ใช้เวลาประมวลผลกี่วินาที			
32	ดูกราฟหุ้นใน HERO ได้ที่ไหน			
33	ปรับรูปแบบกราฟแท่งเทียนยังไง			
34	มีอินดิเคเตอร์ให้เลือกไหม เช่น RSI, MACD			
35	สามารถวาดเส้นแนวรับแนวต้านใน HERO ได้ไหม			
36	ดู Timeframe แบบ 1 นาที ได้ไหม			
37	ตั้งกราฟหลายหน้าจอสองพร้อมกันได้ไหม			
38	ดูกราฟย้อนหลังได้กี่ปี			
39	โหลดข้อมูลกราฟออกไปใช้ Excel ได้ไหม			
40	กราฟใน HERO เป็นเรียลไทม์ไหม			
41	ดู Volume Profile ได้ไหมใน HERO			
42	ตั้งแจ้งเตือนราคาหุ้นทั้งใจ			

รูปที่ ก.1 เป็นชุดคำถามตัวอย่างที่จัดทำขึ้นเพื่อใช้ในการทดสอบและฝึกสอนระบบ Chatbot ของ Finansia HERO โดยคำถามถูกออกแบบให้ครอบคลุมหลายหมวดหมู่ เพื่อประเมินความสามารถของ Chatbot ในการตอบคำถามของผู้ใช้งานจริงได้อย่างถูกต้องและเป็นธรรมชาติ

ภาคผนวก ข

ภาพรวมโค้ดและโครงสร้างโปรเจกต์ (Visual Studio Code)

ภาคผนวก ข



รูปที่ ข.1 โครงสร้าง Directory และไฟล์ของโปรเจกต์ Chatbot
(ซึ่งแบ่งเป็นส่วนหลักคือ backend และ frontend)

```

# --- Node 1: router (decision) ---
def router_node(state: AgentState, config: RunnableConfig) -> AgentState:
    print("\n--- Entering router_node ---")

    # Step 1: Extract original user query
    original_query = next((m.content for m in reversed(state["messages"]) if isinstance(m, HumanMessage)), "")
    print(f"Original user query: {original_query}")

    # Step 2: Use original query for routing (enhancement moved to RAG/web nodes)
    query = original_query # Router uses original query for classification

    print(f"✓ Router using original query (no enhancement): {query}")

    # Step 3: Get web search configuration
    web_search_enabled = config.get("configurable", {}).get("web_search_enabled", True)
    force_web_search = config.get("configurable", {}).get("force_web_search", False)
    print(f"Web search enabled: {web_search_enabled}")
    print(f"Force web search: {force_web_search}")

    # Step 4: Simplified Router Prompt (Optimized for speed and clarity)
    system_prompt = f"""You are {BOT_NAME}, a routing agent for {DOMAIN_NAME}.
    You specialize in: {' '.join(SPECIALTY_AREAS[:3])}, and more.

    Route to ONE of these:"""

    if web_search_enabled:
        system_prompt += """
        - 'rag': Platform features, trading procedures, account help, troubleshooting
        - 'web': Current market data, prices, financial news (trusted sources only)
        - 'answer': Simple greetings or basic factual info (contact, social media, capabilities)
        - 'end': Chitchat or completely off-topic queries (provide 'reply')"""
    else:
        system_prompt += """
        - 'rag': ALL platform/trading questions AND any market info queries
        - 'answer': Simple greetings or basic factual info (contact, social, capabilities)
        - 'end': Chitchat or completely off-topic queries (provide 'reply')
        **NOTE: Web search is DISABLED - route market data queries to 'rag'*****

        system_prompt += f"""

    Examples:
    - "How to set stop loss?" → 'rag'
    - "Bitcoin price now?" → '{"web" if web_search_enabled else "rag"}'
    - "Your phone number?" → 'answer' (is_basic_info=True)
    - "Hello!" → 'end' (reply='Hello! I'm {BOT_NAME}. How can I help?')
    - "Tell me a joke" → 'end' (out_of_bounds=True, reply='I focus on {DOMAIN_NAME}. Ask me about trading!')

    **OUT-OF-BOUNDS DETECTION (3-step decision tree):**

    Step 1: Domain Check - Does query mention finance/trading/business/platform?
    ✓ YES → Continue to Step 2
    ✗ NO → Likely out-of-bounds, go to Step 3

```

รูปที่ ข.2 ตัวอย่างตรงส่วน Core Agent Logic
(Router Node หรือ ตรรกะการจัดเส้นทาง query)

```

import os
from dotenv import load_dotenv

load_dotenv() # Load environment variables from .env file

# === NEW: Qdrant Vector Database Configuration ===
QDRANT_API_KEY = os.getenv("QDRANT_API_KEY")
QDRANT_URL = os.getenv("QDRANT_URL")
QDRANT_COLLECTION_NAME = os.getenv("QDRANT_COLLECTION_NAME", "fsshero-chatbot-bge-m3")

# === NEW: Google/Gemini Configuration ===
GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY")

# === EXISTING: Tavily (keeping for parity) ===
TAVILY_API_KEY = os.getenv("TAVILY_API_KEY")

# === NEW: BGE-M3 Embedding Model ===
EMBED_MODEL = os.getenv("EMBED_MODEL", "BAAI/bge-m3")
EMBED_DIMENSIONS = 1024 # BGE-M3 embedding dimensions

# === LEGACY: Old configurations (commented for potential rollback) ===
# PINECONE_API_KEY = os.getenv("PINECONE_API_KEY")
# PINECONE_ENVIRONMENT = os.getenv("PINECONE_ENVIRONMENT", "us-east-1")
# PINECONE_INDEX_NAME = os.getenv("PINECONE_INDEX_NAME", "rag-index")
# GROQ_API_KEY = os.getenv("GROQ_API_KEY")
# OLD_EMBED_MODEL = "sentence-transformers/all-MiniLM-L6-v2" # 384 dimensions

# === EXISTING: Paths ===
DOC_SOURCE_DIR = os.getenv("DOC_SOURCE_DIR", "data")

# === NEW: HERO Bot Domain-Specific Configuration ===
DOMAIN_NAME = os.getenv("DOMAIN_NAME", "Finansia Hero Trading Platform")
BOT_NAME = os.getenv("BOT_NAME", "HERO Bot")
SEARCH_DOMAINS = os.getenv("SEARCH_DOMAINS", "www.finansiahero.com,smartaccess.fnsyurus.com/open-board/").split(',')

# HERO Bot Specialization Areas (identical to original HERO Bot)
SPECIALTY_AREAS = [
    "Platform features and navigation",
    "Trading tools and order management",
    "Technical analysis and charting",
    "Account management and settings",
    "Risk management and trading strategies",
    "Troubleshooting platform issues"
]

# HERO Bot Response Settings
DEFAULT_SIMILARITY_THRESHOLD = float(os.getenv("DEFAULT_SIMILARITY_THRESHOLD", "0.7"))
FINANCIAL_TEMPERATURE = float(os.getenv("FINANCIAL_TEMPERATURE", "0.3"))
ENABLE_QUERY_ENHANCEMENT = os.getenv("ENABLE_QUERY_ENHANCEMENT", "true").lower() == "true"

# Unified Language Policy
LANGUAGE_POLICY = """**LANGUAGE POLICY:**
- Match the user's language: if they ask in Thai, respond in Thai; if in English, respond in English
- Preserve language consistency throughout the conversation
- Do NOT translate or change the user's language unless explicitly requested"""

```

รูปที่ ข.3 ตัวอย่างตรงส่วน Configuration Management
(API Keys & Database Config)

```

import os
from typing import List, Dict, Any
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams
from langchain_qdrant import QdrantVectorStore
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_core.embeddings import Embeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Import configuration - Updated for new structure
from src.backend.core.config import QDRANT_API_KEY, QDRANT_URL, QDRANT_COLLECTION_NAME, EMBED_MODEL, EMBED_DIMENSIONS

class BGEEEmbedder(Embeddings):
    """BGE-M3 embeddings wrapper for better multilingual retrieval"""

    def __init__(self, model_name: str = None):
        """Initialize BGE-M3 embeddings model."""
        model_name = model_name or EMBED_MODEL
        print(f"📦 Loading BGE-M3 model: {model_name}")
        self.embeddings = HuggingFaceEmbeddings(
            model_name=model_name,
            model_kwargs={'device': 'cpu'}, # Use CPU for compatibility
            encode_kwargs={'normalize_embeddings': True} # Normalize for better retrieval
        )
        print("✅ BGE-M3 model loaded successfully")

    def embed_documents(self, texts: List[str]) -> List[List[float]]:
        """Embed a list of documents."""
        return self.embeddings.embed_documents(texts)

    def embed_query(self, text: str) -> List[float]:
        """Embed a single query."""
        return self.embeddings.embed_query(text)

# Initialize embeddings instance
embeddings = BGEEEmbedder()

# Initialize Qdrant client
def _get_qdrant_client():
    """Get Qdrant client instance"""
    if not all([QDRANT_API_KEY, QDRANT_URL]):
        raise ValueError("QDRANT_API_KEY and QDRANT_URL must be set in environment variables")

    try:
        client = QdrantClient(url=QDRANT_URL, api_key=QDRANT_API_KEY, timeout=60)
        return client
    except Exception as e:
        raise Exception(f"Failed to connect to Qdrant: {str(e)}")

# --- Retriever (Compatible with existing interface) ---
def get_retriever():
    """Initializes and returns the Qdrant vector store retriever."""
    client = _get_qdrant_client()

```

รูปที่ ข.4 ตัวอย่างตรงส่วน Vector Store Operations
(BGE-M3 Embedder Class การสร้าง embeddings)

```

# --- Chat Endpoint ---
@app.post("/chat/", response_model=AgentResponse)
async def chat_with_agent(request: QueryRequest):
    trace_events_for_frontend: List[TraceEvent] = []

    try:
        # Pass enhanced parameters into the config for the agent to access
        config = {
            "configurable": {
                "thread_id": request.session_id,
                "web_search_enabled": request.enable_web_search,
                "force_web_search": request.force_web_search,
                "similarity_threshold": request.similarity_threshold
            }
        }
        inputs = {"messages": [HumanMessage(content=request.query)]}

        final_message = ""

        print(f"--- Starting Agent Stream for session {request.session_id} ---")
        print(f"Web Search Enabled: {request.enable_web_search}") # For server-side debugging
        print(f"Force Web Search: {request.force_web_search}") # For server-side debugging
        print(f"Similarity Threshold: {request.similarity_threshold}") # For server-side debugging

        for i, s in enumerate(rag_agent.stream(inputs, config=config)):
            current_node_name = None
            node_output_state = None

            if '__end__' in s:
                current_node_name = '__end__'
                node_output_state = s['__end__']
            else:
                current_node_name = list(s.keys())[0]
                node_output_state = s[current_node_name]

            event_description = f"Executing node: {current_node_name}"
            event_details = {}
            event_type = "generic_node_execution"

            if current_node_name == "router":
                route_decision = node_output_state.get('route')
                # Check for overridden route if web search was disabled
                initial_decision = node_output_state.get('initial_router_decision', route_decision)
                override_reason = node_output_state.get('router_override_reason', None)

                # NEW: HERO Bot Query Enhancement Information
                original_query = node_output_state.get('original_query', request.query)
                enhanced_query = node_output_state.get('enhanced_query', request.query)
                query_enhanced = enhanced_query != original_query and node_output_state.get('query_enhancement_enabled', False)

                if query_enhanced:
                    event_description = f"HERO Bot enhanced query and decided: '{route_decision}'"
                    event_details = {
                        "decision": route_decision
                    }

```

รูปที่ ข.5 ตัวอย่างตรงส่วน FastAPI Endpoints
(Chat Endpoint หลักของระบบ chatbot)

```

# --- Build graph ---
def build_agent():
    """Builds and compiles the LangGraph agent with dedicated enhancement node."""
    g = StateGraph(AgentState)
    g.add_node("router", router_node)
    g.add_node("enhance", enhance_query_node) # NEW: Dedicated enhancement node
    g.add_node("rag_lookup", rag_node)
    g.add_node("web_search", web_node)
    g.add_node("answer", answer_node)

    g.set_entry_point("router")

    # Router can go to: enhance (for rag/web), answer (for basic_info), or END
    g.add_conditional_edges(
        "router",
        from_router,
        {
            "enhance": "enhance", # rag/web routes go through enhancement
            "answer": "answer",    # basic_info skips enhancement
            "end": END             # greetings/oos end immediately
        }
    )

    # Enhancement node routes to rag or web based on router's decision
    g.add_conditional_edges(
        "enhance",
        from_enhance,
        {
            "rag": "rag_lookup",
            "web": "web_search"
        }
    )

    # RAG can go to answer or web (if RAG insufficient)
    g.add_conditional_edges(
        "rag_lookup",
        after_rag,
        {
            "answer": "answer",
            "web": "web_search"
        }
    )

    # Web always goes to answer
    g.add_edge("web_search", "answer")

    # Answer always ends
    g.add_edge("answer", END)

    agent = g.compile(checkpointer=MemorySaver())
    return agent

rag_agent = build_agent()

```

รูปที่ ข.5 ตัวอย่างตรงส่วน Graph Structure Diagram

ประวัติย่อผู้ทำรายงาน

ชื่อ ชื่อสกุล

รหัสนักศึกษา

สถานที่อยู่ปัจจุบัน

ประวัติการศึกษา

ระดับมัธยมศึกษาตอนปลาย

วิทยาลัยรังสิต

ระดับมัธยมศึกษาตอนปลาย

ปริญญาตรี

นาย อชิรธีร์ อินยา

650910304

124/84 หมู่ที่ 1

ต.บึงยี่โถ อ.ธัญบุรี

จ.ปทุมธานี 12130

โรงเรียนสวนกุหลาบ

โรงเรียนหอวัง ปทุมธานี

มหาวิทยาลัย ศิลปากร

